Glove & RNN

Lect4

Glove

What is GloVe?

- Global Vectors for Word Representation.
- It is an unsupervised learning algorithm that generates vector representations, or embeddings, of words.
- Researchers Richard Socher, Christopher D. Manning, and Jeffrey Pennington first presented it in 2014.
- By using the statistical co-occurrence data of words in a given corpus, GloVe is intended to capture the semantic relationships between words.

- The fundamental concept underlying GloVe is the representation of words as vectors in a continuous vector space, where the angle and direction of the vectors correspond to the semantic connections between the appropriate words.

What should I do ?

- GloVe builds a co-occurrence matrix using word pairs and then optimizes the word vectors to minimize the difference between the pointwise mutual information of the corresponding words and the dot product of vectors.

- The glove has pre-defined dense vectors for around every 6 billion words of English along with many other general-use characters like commas, braces, and semicolons.
- The algorithm’s developers frequently make the pre-trained GloVe embeddings available.
- Users can select a pre-trained GloVe embedding in a dimension (e.g., 50-d, 100-d, 200-d, or 300-d vectors) that best fits their needs in terms of computational resources and task specificity.

- Here d stands for dimension. 100d means, in this file each word has an equivalent vector of size 100.
- Glove files are simple text files in the form of a dictionary. Words are key and dense vectors are values of key.

https://github.com/stanfordnlp/GloVe

Download pre-trained word vectors

The links below contain word vectors obtained from the respective corpora. If you want word vectors trained on massive web datasets, you need only download one of these text files! Pre-trained word vectors are made available under the Public Domain Dedication and License.

- Common Crawl (42B tokens, 1.9M vocab, uncased, 300d vectors, 1.75 GB download): glove.42B.300d.zip [mirror]
- Common Crawl (840B tokens, 2.2M vocab, cased, 300d vectors, 2.03 GB download): glove.840B.300d.zip [mirror]
- Wikipedia 2014 + Gigaword 5 (6B tokens, 400K vocab, uncased, 300d vectors, 822 MB download): glove.6B.zip [mirror]
- Twitter (2B tweets, 27B tokens, 1.2M vocab, uncased, 200d vectors, 1.42 GB download): glove.twitter.27B.zip [mirror]

```
the 0.418 0.24968 -0.41242 0.1217 0.34527 -0.044457 -0.49688 -0.17862 -0.00066023 -0.6566 0.27843 -0.14767 -0.55677 0.14658 -0.0095095 0.01165
, 0.013441 0.23682 -0.16899 0.40951 0.63812 0.47709 -0.42852 -0.55641 -0.364 -0.23938 0.13001 -0.063734 -0.39575 -0.48162 0.23291 0.090201 -0.
to 0.68047 -0.039263 0.30186 -0.17792 0.42962 0.032246 -0.41376 0.13228 -0.29847 -0.085253 0.17118 0.22419 -0.10046 -0.43653 0.33418 0.67846 0.05720
and 0.26818 0.14346 -0.27877 0.016257 0.11384 0.69923 -0.51332 -0.47368 -0.33075 -0.13834 0.2702 0.30938 -0.45012 -0.4127 -0.09932 0.038085 0.029749
in 0.33042 0.24995 -0.60874 0.10923 0.036372 0.151 -0.55083 -0.074239 -0.092307 -0.32821 0.09598 -0.82269 -0.36717 -0.67009 0.42909 0.016496 -0.2357
a 0.21705 0.46515 -0.46757 0.10082 1.0135 0.74845 -0.53104 -0.26256 0.16812 0.13182 -0.24909 -0.44185 -0.21739 0.51004 0.13448 -0.43141 -0.03123 0.2
" 0.25769 0.45629 -0.76974 -0.37679 0.59272 -0.063527 0.20545 -0.57385 -0.29009 -0.13662 0.32728 1.4719 -0.73681 -0.12036 0.71354 -0.46098 0.65248 0
's 0.23727 0.40478 -0.20547 0.58805 0.65533 0.32867 -0.81964 -0.23236 0.27428 0.24265 0.054992 0.16296 -1.2555 -0.086437 0.44536 0.096561 -0.16519 0
for 0.15272 0.36181 -0.22168 0.066051 0.13029 0.37075 -0.75874 -0.44722 0.22563 0.10208 0.054225 0.13494 -0.43052 -0.2134 0.56139 -0.21445 0.077974
- -0.16768 1.2151 0.49515 0.26836 -0.4585 -0.23311 -0.52822 -1.3557 0.16098 0.37691 -0.92702 -0.43904 -1.0634 1.028 0.0053943 0.04153 -0.018638 -0.5
that 0.88387 -0.14199 0.13566 0.098682 0.51218 0.49138 -0.47155 -0.30742 0.01963 0.12686 0.073524 0.35836 -0.60874 -0.18676 0.78935 0.54534 0.1106 -
on 0.30045 0.25006 -0.16692 0.1923 0.026921 -0.079486 -0.91383 -0.1974 -0.053413 -0.40846 -0.26844 -0.28212 -0.5 0.1221 0.3903 0.17797 -0.4429 -0.40
is 0.6185 0.64254 -0.46552 0.3757 0.74838 0.53739 0.0022239 -0.60577 0.26408 0.11703 0.43722 0.20092 -0.057859 -0.34589 0.21664 0.58573 0.53919 0.69
was 0.086888 -0.19416 -0.24267 -0.33391 0.56731 0.39783 -0.97809 0.03159 -0.61469 -0.31406 0.56145 0.12886 -0.84193 -0.46992 0.47097 0.023012 -0.596
said 0.38973 -0.2121 0.51837 0.80136 1.0336 -0.27784 -0.84525 -0.25333 0.12586 -0.90342 0.24975 0.22022 -1.2053 -0.53771 1.0446 0.62778 0.39704 -0.1
with 0.25616 0.43694 -0.11889 0.20345 0.41959 0.85863 -0.60344 -0.31835 -0.6718 0.003984 -0.075159 0.11043 -0.73534 0.27436 0.054015 -0.23828 -0.137
he -0.20092 -0.060271 -0.61766 -0.8444 0.5781 0.14671 -0.86098 0.6705 -0.86556 -0.18234 0.15856 0.45814 -1.0163 -0.35874 0.73869 -0.24048 -0.33893 0
as 0.20782 0.12713 -0.30188 -0.23125 0.30175 0.33194 -0.52776 -0.44042 -0.48348 0.03502 0.34782 0.54574 -0.2066 -0.083713 0.2462 0.15931 -0.0031349
```

How GloVe works?

- Preprocess the text data.
- Create the dictionary.
- Traverse the glove file of a specific dimension and compare each word with all words in the dictionary
- If a match occurs, copy the equivalent vector from the glove and paste into embedding_matrix at the corresponding index

How GloVe works?

Create Vocabulary Dictionary:

- Vocabulary is the collection of all unique words present in the training dataset.
- The dataset is tokenized into words, then all the frequency of each word is counted.
- Then words are sorted in decreasing order of their frequencies.
- Words having high frequency are placed at the beginning of the dictionary.

Dataset= {The peon is ringing the bell}
Vocab= {'The':2, 'peon':1, 'is':1, 'ringing':1}

```
umber of unique words in dictionary= 6
ictionary is =  {'leader': 1, 'the': 2, 'prime': 3, 'natural': 4, 'language': 5, 'text': 6}
ense vector for first word is =>  [-0.1567      0.26117     0.78881001  0.65206999  1.20019996
.35400999
-0.34298     0.31702    -1.15020001 -0.16099     0.15798    -0.53501999
-1.34679997  0.51783001 -0.46441001 -0.19846     0.27474999 -0.26154
 0.25531     0.33388001 -1.04130006  0.52525002 -0.35442999 -0.19137
-0.08964    -2.33139992  0.12433    -0.94405001 -1.02330005  1.35070002
 2.55240011 -0.16897    -1.72899997  0.32548001 -0.30914    -0.63056999
-0.22211    -0.15589    -0.43597999  0.0568     -0.090885    0.75028002
-1.31529999 -0.75358999  0.82898998  0.051397   -1.48049998 -0.11134
 0.27090001 -0.48712999]
```

	Passport	Exams	Explore	Money
P(k/Travel)	0.25	0.004	0.26	0.45
P(k/School)	0.003	0.34	0.28	0.41
P(k/Travel) / P(k/School)	83	0.01	0.92	1.09

Probability and Ratio	$k = solid$	$k = gas$	$k = water$	$k = fashion$
$P(k\|ice)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k\|steam)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k\|ice)/P(k\|steam)$	8.9	8.5×10^{-2}	1.36	0.96

"I like deep learning. I like NLP. I enjoy flying."

	I	like	deep	learning	NLP	enjoy	flying
I	0	2	0	0	0	1	0
like	2	0	1	0	1	0	0
deep	0	1	0	1	0	0	0
learning	0	0	1	0	0	0	0
NLP	0	1	0	0	0	0	0
enjoy	1	0	0	0	0	0	1
flying	0	0	0	0	0	1	0

- Probability ratios table

$$P_{ij} = \frac{X_{ij}}{\sum_k X_{ik}}$$

where P_{ij} is the probability of word j occurring in the context of word i.

Now, we compute probabilities:

	I	like	deep	learning	NLP	enjoy	flying
I	0	2/3	0	0	0	1/3	0
like	2/4	0	1/4	0	1/4	0	0
deep	0	1/2	0	1/2	0	0	0
learning	0	0	1/1	0	0	0	0
NLP	0	1/1	0	0	0	0	0
enjoy	1/2	0	0	0	0	0	1/2
flying	0	0	0	0	0	1/1	0

The probability ratio of words w_j and w_k in the context of w_i is:

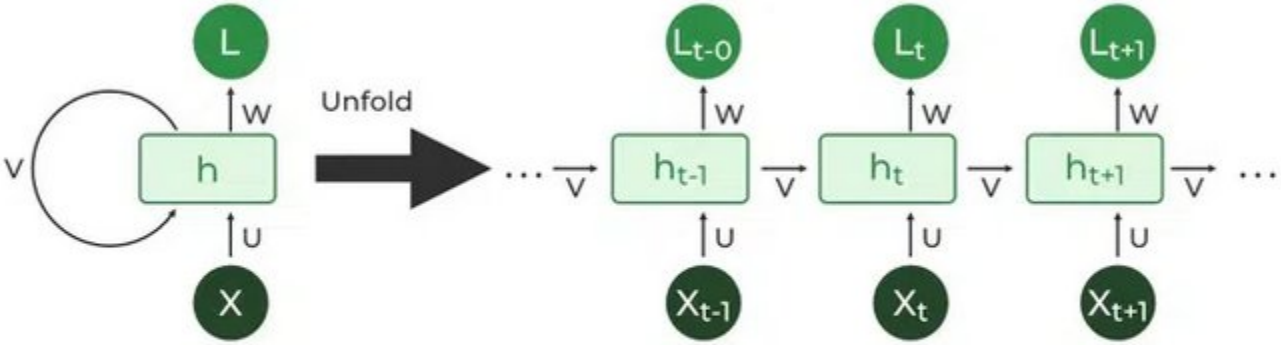
$$\frac{P_{ij}}{P_{ik}}$$

	I	like	deep	learning	NLP	enjoy	flying
I	-	$\frac{\frac{2}{3}}{\frac{1}{3}} = 2$	-	-	-	$\frac{\frac{1}{3}}{\frac{2}{3}} = 0.5$	-
like	$\frac{\frac{2}{4}}{\frac{1}{4}} = 2$	-	$\frac{\frac{1}{4}}{\frac{2}{4}} = 0.5$	-	$\frac{\frac{1}{4}}{\frac{2}{4}} = 0.5$	-	-
deep	-	$\frac{\frac{1}{2}}{\frac{1}{2}} = 1$	-	$\frac{\frac{1}{2}}{\frac{1}{2}} = 1$	-	-	-
learning	-	-	$\frac{\frac{1}{1}}{\frac{1}{1}} = 1$	-	-	-	-
NLP	-	$\frac{\frac{1}{1}}{\frac{1}{1}} = 1$	-	-	-	-	-
enjoy	$\frac{\frac{1}{2}}{\frac{1}{2}} = 1$	-	-	-	-	-	$\frac{\frac{1}{2}}{\frac{1}{2}} = 1$
flying	-	-	-	-	-	$\frac{\frac{1}{1}}{\frac{1}{1}} = 1$	-

RNN

Recurrent Neural Networks (RNNs) work a bit different from regular neural networks.
In neural network the information flows in one direction from input to output.
However in RNN information is fed back into the system after each step.

- RNNs allow the network to “remember” past information by feeding the output from one step into next step.
- This helps the network understand the context of what has already happened and make better predictions based on that.



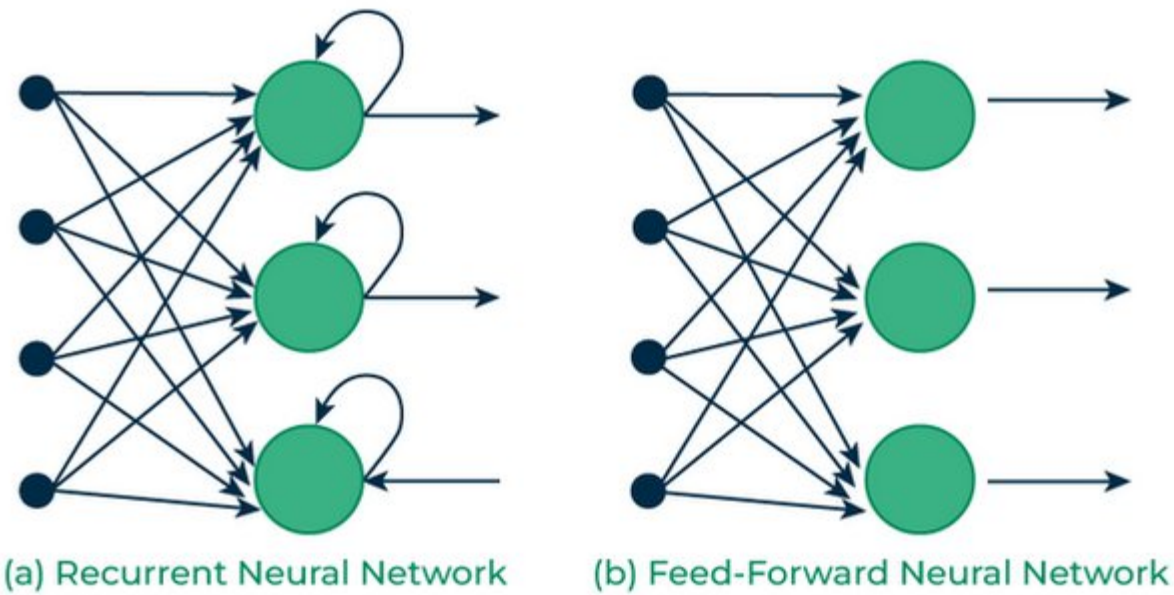
Recurrent Neural Network

How RNN Differs from Feedforward Neural Networks?

How RNN Differs from Feedforward Neural Networks?

- Feedforward Neural Networks (FNNs) process data in one direction from input to output without retaining information from previous inputs. This makes them suitable for tasks with independent inputs like image classification. However FNNs struggle with sequential data since they lack memory.

- Recurrent Neural Networks (RNNs) solve this **by incorporating loops that allow information from previous steps to be fed back into the network**. This feedback enables RNNs to remember prior inputs making them ideal for tasks where context is important.



Recurrent Vs Feedforward networks

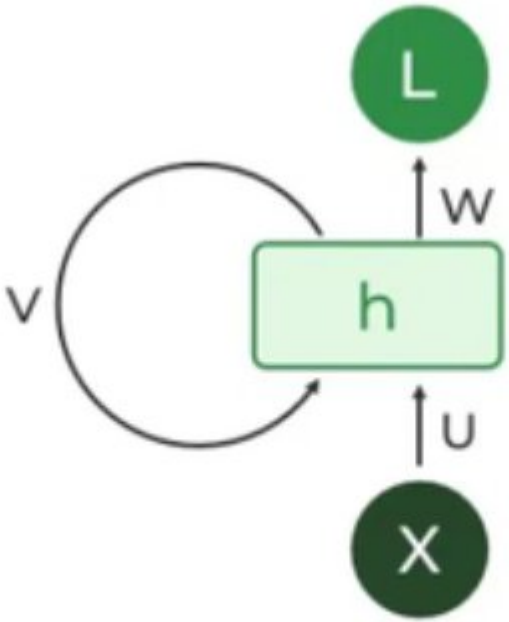
Key Components of RNNs

Key Components of RNNs

- **1. Recurrent Neurons**
- **2. RNN Unfolding**

Recurrent Neurons

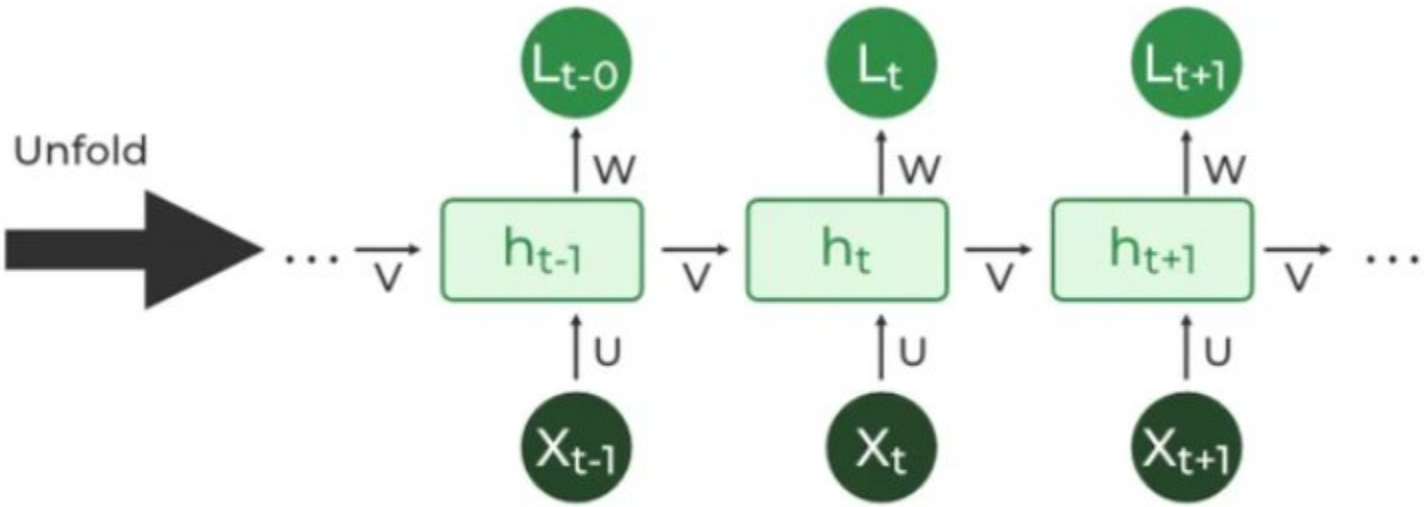
- The fundamental processing unit in RNN is a **Recurrent Unit.** Recurrent units hold a hidden state that maintains information about previous inputs in a sequence. Recurrent units can “remember” information from prior steps by feeding back their hidden state, allowing them to capture dependencies across time.



Recurrent Neuron

RNN Unfolding

- RNN unfolding or unrolling is the process of expanding the recurrent structure over time steps. During unfolding each step of the sequence is represented as a separate layer in a series illustrating how information flows across each time step.



RNN Unfolding

How does RNN work?

- At each time step RNNs process units with a fixed activation function. These units have an internal hidden state that acts as memory that retains information from previous time steps. This memory allows the network to store past knowledge and adapt based on new inputs.

- **Updating the Hidden State in RNNs**
- The current hidden state $h_t$depends on the previous state h_{t-1} and the current input x_t, and is calculated using the following relations:

1. State Update:

$$h_t = f(h_{t-1}, x_t)$$

where:

- h_t is the current state
- h_{t-1} is the previous state
- x_t is the input at the current time step

- **Activation Function Application**

$$h_t = f\ (w_{hh} \cdot h_{t-1} + w_{xh} . x_t$$

Here, W_{hh} is the weight matrix for the recurrent neuron, and W_{xh} is the weight matrix for the input neuro

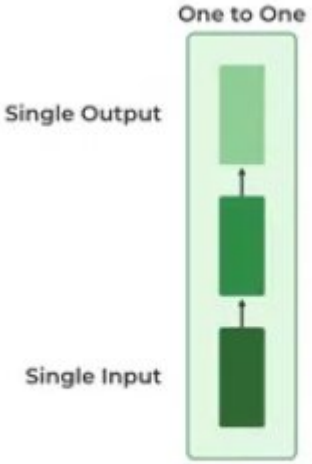
- **3. Output Calculation:**

$$y_t = W_{hy} \cdot h_t$$

where y_t is the output and W_{hy} is the weight at the output layer.

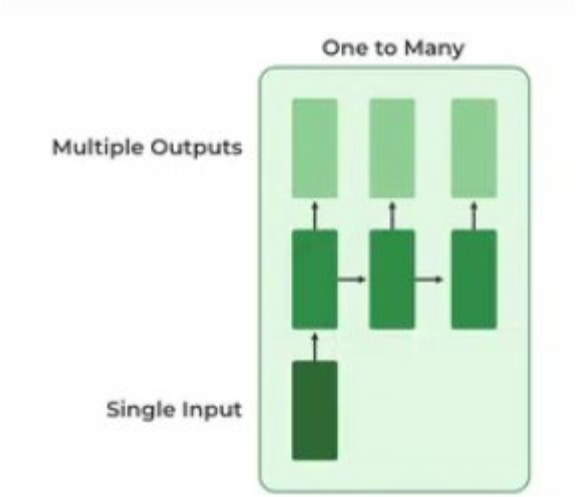
Types Of Recurrent Neural Networks

- **One-to-One RNN**



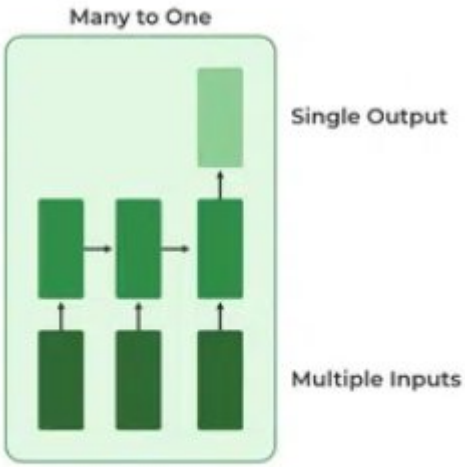
One to One RNN

- **One-to-Many RNN**



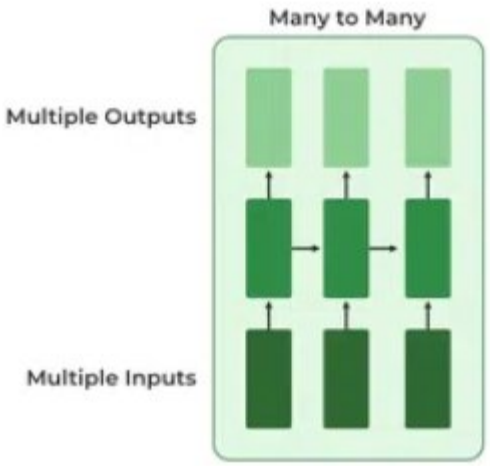
One to Many RNN

- **Many-to-One RNN**

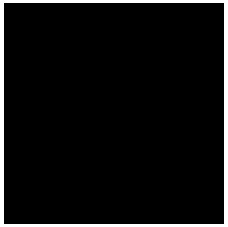


Many to One RNN

- **Many-to-Many RNN**



Many to Many RNN

Variants of Recurrent Neural Networks (RNNs)

- **Vanilla RNN**
- **Bidirectional RNNs**
- **Long Short-Term Memory Networks (LSTMs)**
- **Gated Recurrent Units (GRUs)**